

FitBuddy – AI Fitness Plan Generator using Gemini Models

Project Description:

FitBuddy is a web-based application that uses AI to generate personalized workout plans and nutrition tips based on a user's fitness goal, such as weight loss, muscle gain, or general wellness. It is built with FastAPI and integrated with Google's Gemini AI models, which removes the effort of creating a structured and effective fitness routine by hand.

The application takes user information such as name, user ID, age, weight, fitness goal and preferred workout intensity, and generates a customized 7-day fitness plan. Users can also submit feedback (for example, "add more cardio") to update and refine the plan using AI. A short nutrition or recovery tip is also provided based on the selected goal.

With a clean, responsive interface and database support through SQLite and SQLAlchemy, FitBuddy stores every user's data and generated plans. The original plan is never overwritten: the feedback-based update is saved separately so both versions can be compared. The project offers an interactive HTML interface, an admin dashboard, and Swagger API documentation at `/docs`.

Scenarios:

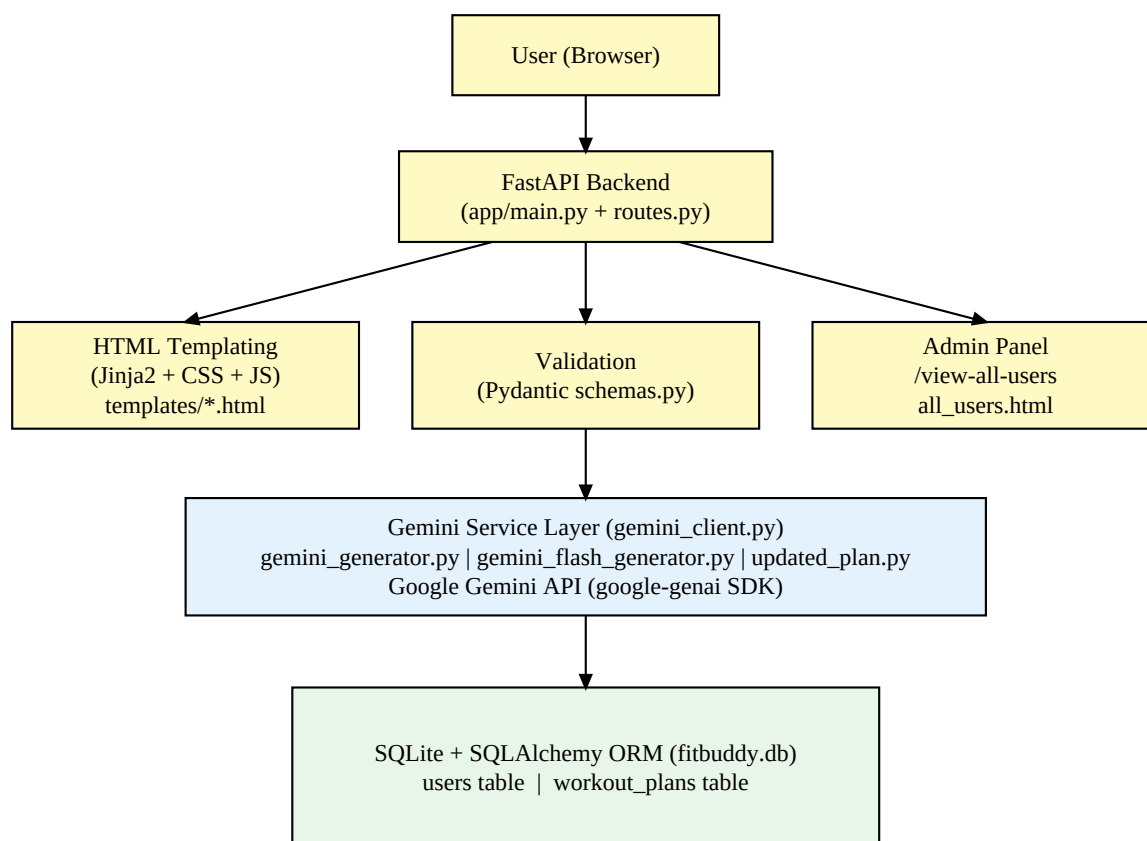
Scenario 1: A user enters name, user ID, age, weight, fitness goal (Weight Loss, Muscle Gain, General Wellness, Flexibility, Strength or Endurance) and workout intensity (Low, Medium or High). FitBuddy generates a day-by-day 7-day plan with focus area, warm-up, exercises, sets/reps/duration, rest and cooldown for each day, tailored to the goal and intensity.

Scenario 2: A user submits feedback on the existing plan, such as "more focus on cardio" or "include more rest days". The system sends the original plan and the feedback to Gemini, which returns a revised plan. Both the original and the updated plan are stored.

Scenario 3: Along with the plan, the platform delivers a short nutrition or recovery tip for the chosen goal (for example, protein advice for muscle gain). The tip is generated by a lighter Gemini model for a fast response. If only the tip fails, the workout plan is still saved and shown.

Scenario 4: An admin or coach opens the dashboard at `/view-all-users` to see all users, statistics (total users, total plans, updated plans, goal distribution), search by name or user ID, and compare original and updated plans.

Technical Architecture:



Pre-requisites:

No.	Skill / Tool	Reference
1	FastAPI framework knowledge	https://fastapi.tiangolo.com
2	Gemini API familiarity (google-genai SDK)	https://ai.google.dev/gemini-api/docs
3	HTML, CSS and Jinja2 templates	https://jinja.palletsprojects.com
4	Python programming (3.10 or newer)	https://docs.python.org/3/
5	Version control with Git	https://git-scm.com/doc
6	SQLAlchemy and SQLite basics	https://docs.sqlalchemy.org
7	Environment setup (pip, venv)	https://docs.python.org/3/library/venv.html
8	Uvicorn ASGI server	https://www.uvicorn.org

Project Workflow:

Milestone 1. Model selection and Architecture

- Activity 1.1. Research and select the appropriate generative AI model
- Activity 1.2. Define the architecture of the application
- Activity 1.3. Set up the development environment

Milestone 2. Core functionalities development

- Activity 2.1. Develop the core functionalities
- Activity 2.2. Implement the FastAPI backend to manage routing and user input processing

Milestone 3. routes.py development

- Activity 3.1. Writing the main application logic in routes.py

Milestone 4. Frontend development

- Activity 4.1. Designing and developing the user interface
- Activity 4.2. Creating dynamic templates with FastAPI's Jinja2

Milestone 5. Deployment

- Activity 5.1. Preparing the application for local deployment
- Activity 5.2. Testing and verifying local deployment

Milestone 1. Model Selection and Architecture

In this milestone the appropriate generative AI model from Google's Gemini family is selected and the overall architecture is defined. The models must generate personalized workout plans, update them from feedback, and provide short nutrition or recovery advice.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Project requirements

- Generate structured 7-day workout plans
- Provide concise and actionable nutrition tips
- Update workout plans based on user feedback
- Respond fast enough for real-time, web-based interaction

Model evaluation

Open-source Hugging Face models such as T5, BART, DistilBERT and Bloom were considered first. They need local GPU hosting or fine-tuning and complex prompting to produce structured plans. Google's Gemini models are used instead because they are API based, easy to integrate with FastAPI, follow structured output formats reliably and need minimal prompt tuning.

Final model selection

Role	Used for	Environment variable	Configured value
Workout model	7-day plan generation and feedback-based plan update	GEMINI_WORKOUT_MODEL	gemini-2.5-flash
Tip model	Short nutrition / recovery tip (fast, light call)	GEMINI_TIP_MODEL	gemini-2.5-flash
Fallback models	Tried automatically if a model returns 404 / unavailable	GEMINI_FALLBACK_MODELS	gemini-2.5-flash

Model names are not hard-coded. Google retires Gemini model names from time to time (for example, Gemini 1.5 is no longer available), so the application reads model names from the `.env` file. Any current Gemini model can be used without editing the code. Two logical roles are kept: one model for plan generation and revision, and one light model for the tip.

Activity 1.2: Define the Architecture of the Application

FitBuddy follows a modular FastAPI-based architecture:

- **Frontend:** HTML + CSS + vanilla JavaScript rendered through Jinja2 templates
- **Backend:** FastAPI for routing and logic, Pydantic for validation
- **AI layer:** Google Gemini API through one centralized service module (`gemini_client.py`)
- **Database:** SQLite with SQLAlchemy ORM (`fitbuddy.db`, created automatically)

Frontend responsibilities

- `base.html` – common layout inherited by all pages
- `index.html` – input form
- `result.html` – plan, tip and feedback form
- `feedback.html` – dedicated feedback page
- `all_users.html` – admin dashboard
- `error.html` – friendly error page

Backend responsibilities

- Receive and validate form data
- Call the Gemini models
- Store user information and plans
- Render dynamic templates and handle errors gracefully

AI integration points

- Workout generation and feedback updates -> workout model
- Nutrition / recovery tip -> tip model

Activity 1.3: Set up the Development Environment

Install Python and pip: Python 3.10 or newer must be installed and added to PATH.

Create and activate the virtual environment (Windows):

```
python -m venv venv
venv\Scripts\activate
```

Install required libraries:

```
pip install -r requirements.txt
```

requirements.txt contains:

```
fastapi>=0.115,<1.0
uvicorn[standard]>=0.34,<1.0
jinja2>=3.1,<4.0
sqlalchemy>=2.0,<3.0
python-multipart>=0.0.9
python-dotenv>=1.0,<2.0
pydantic>=2.9,<3.0
google-generativeai>=1.0
```

Run the server:

```
uvicorn app.main:app --reload
```

Visit <http://127.0.0.1:8000> for the application and <http://127.0.0.1:8000/docs> for API testing.

Project structure

```
FitBuddy/
|-- requirements.txt          # Project dependencies
|-- run.bat                  # One-click setup + run script (Windows)
|-- .env / .env.example      # API key and model settings
|-- README.md
|-- app/
|   |-- main.py              # FastAPI entry point
|   |-- routes.py            # Core route handlers
|   |-- database.py          # Engine, session, init_db()
|   |-- models.py            # SQLAlchemy models: User, WorkoutPlan
|   |-- schemas.py           # Pydantic models for validation
|   |-- gemini_client.py     # Centralized Gemini service layer
|   |-- gemini_generator.py  # Workout plan generator
|   |-- gemini_flash_generator.py # Nutrition tip generator
|   |-- updated_plan.py      # Feedback-based plan updater
|-- templates/
|   |-- base.html, index.html, result.html
|   |-- feedback.html, all_users.html, error.html
|-- static/
|   |-- css/style.css        # Styling
|   |-- js/script.js         # Loading-state button behaviour
|-- fitbuddy.db              # SQLite database (auto-generated)
```

Milestone 2. Core Functionalities Development

Activity 2.1: Develop the Core Functionalities

Workout Plan Generation

Function: `generate_workout_gemini()`

File: `app/gemini_generator.py`

This function builds a prompt from the user's name, age, weight, goal and intensity and sends it to the workout model. Gemini is asked to reply with strict JSON containing exactly 7 day entries, each with day, focus, warm_up, exercises, sets_reps, rest and cooldown. The response is parsed and stored as structured JSON so that the result page can show each day as a separate card.

```
def generate_workout_gemini(name, age, weight, goal, intensity):
    prompt = WORKOUT_PROMPT_TEMPLATE.format(
        name=name, age=age, weight=weight, goal=goal, intensity=intensity)
    raw_text = call_gemini(WORKOUT_MODEL, prompt, want_json=True, temperature=0.8)
    storage_json = plan_to_storage_json(raw_text)
    return storage_json, raw_text
```

Nutrition Tip Generation

Function: `generate_nutrition_tip_with_flash()`

File: `app/gemini_flash_generator.py`

This function generates one short, practical nutrition and recovery tip (3 to 5 sentences, plain text) for the selected goal using the lighter tip model. The prompt tells the model not to give medical advice and not to recommend supplements, medications or extreme diets. The tip is displayed below the workout plan on the result page.

```
def generate_nutrition_tip_with_flash(goal: str) -> str:
    prompt = TIP_PROMPT_TEMPLATE.format(goal=goal)
    tip = call_gemini(TIP_MODEL, prompt, want_json=False, temperature=0.6)
    return tip.strip()
```

Feedback-Based Plan Updating

Function: `update_workout_plan()`

File: `app/updated_plan.py`

The original plan, the user's details and the feedback text are passed to the workout model. The prompt instructs Gemini to actually change the relevant days instead of repeating the old plan, and to return the same 7-day JSON schema. The result is stored as the updated plan.

```
def update_workout_plan(original_plan_json, feedback, name, age, weight, goal, intensity):
    pretty_original = json.dumps(json.loads(original_plan_json), indent=2)
    prompt = UPDATE_PROMPT_TEMPLATE.format(original_plan_json=pretty_original,
        name=name, age=age, weight=weight, goal=goal,
        intensity=intensity, feedback=feedback)
    raw_text = call_gemini(WORKOUT_MODEL, prompt, want_json=True, temperature=0.8)
    return plan_to_storage_json(raw_text), raw_text
```

Centralized Gemini Service Layer

File: `app/gemini_client.py`

All Gemini calls go through this one file. It reads the API key from the environment, creates and caches the client, tries fallback models when a model is unavailable, maps failures to friendly errors (`GeminiConfigError` for a missing or invalid key, `GeminiRequestError` for network, quota or other failures), and safely parses the JSON response. If Gemini returns text that is not valid JSON, the raw text is stored instead so that the application never crashes.

```
def call_gemini(model, prompt, want_json=False, temperature=0.7):
    client = get_client()
    config_kwargs = {"temperature": temperature}
    if want_json:
        config_kwargs["response_mime_type"] = "application/json"
    for candidate in _model_candidates(model): # configured model, then fallbacks
        response = client.models.generate_content(
            model=candidate, contents=prompt,
            config=types.GenerateContentConfig(**config_kwargs))
    ...
    return text
```

Database Models

File: `app/models.py`, `app/database.py`

All user information and AI-generated plans are stored in SQLite through SQLAlchemy ORM. The database file and tables are created automatically when the server starts. There is a one-to-many relationship: one user can have many plans.

Table	Columns
users	id, user_id (unique), name, age, weight, goal, intensity, created_at
workout_plans	id, user_id (foreign key to users.user_id), original_plan, updated_plan, nutrition_tip, feedback, created_at, updated_at

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(String(64), unique=True, index=True, nullable=False)
    name = Column(String(100), nullable=False)
    age = Column(Integer, nullable=False)
    weight = Column(Float, nullable=False)
    goal = Column(String(50), nullable=False)
    intensity = Column(String(20), nullable=False)
    created_at = Column(DateTime, default=utcnow)
    plans = relationship("WorkoutPlan", back_populates="user",
                        cascade="all, delete-orphan")

class WorkoutPlan(Base):
    __tablename__ = "workout_plans"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(String(64), ForeignKey("users.user_id"), nullable=False, index=True)
    original_plan = Column(Text, nullable=False) # JSON text
    updated_plan = Column(Text, nullable=True) # JSON text after feedback
    nutrition_tip = Column(Text, nullable=True)
    feedback = Column(Text, nullable=True)
    created_at = Column(DateTime, default=utcnow)
    updated_at = Column(DateTime, nullable=True)
```

Input Validation

File: `app/schemas.py`

Pydantic models `UserInput` and `FeedbackRequest` validate every form submission before any AI call or database write.

Field	Rule
name	Required, up to 100 characters
user_id	Required, letters, numbers, _ - and . only, up to 64 characters
age	Whole number between 10 and 100
weight	Positive number, up to 400 kg
goal	One of: Weight Loss, Muscle Gain, General Wellness, Flexibility, Strength, Endurance
intensity	One of: Low, Medium, High
feedback	Required, up to 1000 characters

Admin View of All Users

Route: `/view-all-users`

Template: `all_users.html`

Displays every user with their original and updated plans, the feedback given, dashboard statistics (total users, total plans, updated plans, goal distribution) and a search box that matches name or user ID. The page is rendered with a Jinja2 `{% for %}` loop over the rows prepared in `routes.py`.

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Define routes in FastAPI

All routing logic is in `app/routes.py`. The application object is created in `app/main.py`, which loads the `.env` file, creates the database tables at startup (lifespan handler), mounts the `/static` folder and includes the router.

```
app = FastAPI(title="FitBuddy - AI Fitness Plan Generator", version="1.0.0",
              lifespan=lifespan)
app.mount("/static", StaticFiles(directory=str(BASE_DIR / "static")), name="static")
app.include_router(router)
```

Process user input

- HTML form created in `index.html`
- Users enter name, `user_id`, age, weight, goal and intensity
- FastAPI captures inputs using `Form(...)` parameters
- Values are received as text and converted by the Pydantic `UserInput` schema, so an invalid value such as "abc" for age shows a friendly error page instead of a raw 422 response

Integrate Gemini API calls

- `/generate-workout` -> `generate_workout_gemini()` for the plan and `generate_nutrition_tip_with_flash()` for the tip

- `/submit-feedback` -> retrieves the plan and sends original plan + feedback to `update_workout_plan()`

Gemini is always called before the database is written, so a failed AI call never creates or changes a user.

Milestone 3. routes.py Development

Milestone 3 builds the routing logic inside `routes.py`, the core operational layer of FitBuddy. It connects the frontend templates, the AI generation modules and the SQLite database. Each route defines how user input is collected, validated, processed and answered.

Activity 3.1: Writing the Main Application Logic in `routes.py`

Core routes

Method	Endpoint	Purpose	Inputs
GET	/	Home page with the plan form (<code>index.html</code>)	-
POST	/generate-workout	Validate input, call Gemini, save and show the plan	name, user_id, age, weight, goal, intensity
GET	/feedback/{user_id}	Feedback page for an existing user	user_id in path
POST	/submit-feedback	Revise the latest plan with Gemini and save the update	user_id, feedback
GET	/view-all-users	Admin dashboard with statistics and search	optional q
GET	/docs	Swagger UI (auto-generated by FastAPI)	-

/ – Home route

- Returns the homepage (`index.html`) with the goal and intensity options
- No form processing logic

```
@router.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(request, "index.html",
    {"goals": sorted(VAILD_GOALS), "intensities": INTENSITIES})
```

/generate-workout – Plan generator

- Receives user input using FastAPI's `Form(...)`
- Builds and validates a `UserInput` Pydantic model
- Calls `generate_workout_gemini()` for the plan
- Calls `generate_nutrition_tip_with_flash()` for the tip (if this alone fails, the plan is still delivered)
- Saves the user (create or update) and the new plan in one database transaction
- Returns `result.html` with the dynamic data

```
@router.post("/generate-workout", response_class=HTMLResponse)
def generate_workout(request: Request, name: str = Form(...), user_id: str = Form(...),
```

```
        age: str = Form(...), weight: str = Form(...), goal: str = Form(...),
        intensity: str = Form(...), db: Session = Depends(get_db)):
    try:
        data = UserInput(name=name, user_id=user_id, age=age, weight=weight,
                        goal=goal, intensity=intensity)
    except ValidationError as exc:
        return render_error(request, f"Invalid input: {first_error_message(exc)}")

    plan_json, _raw = generate_workout_gemini(
        data.name, data.age, data.weight, data.goal, data.intensity)
    nutrition_tip = generate_nutrition_tip_with_flash(data.goal)

    db_user, db_plan = _save_new_plan(db, data, plan_json, nutrition_tip)
    return _render_result(request, db_user, db_plan)
```

(The code above is shortened for readability; the real file also handles Gemini and database errors with friendly error pages.)

/submit-feedback – Update plan with feedback

- Captures the feedback and user_id and validates them with FeedbackRequest
- Finds the user and the user's most recent plan
- Uses update_workout_plan() to revise the original plan via Gemini
- Saves the revised plan as updated_plan and the feedback text (the original plan is never overwritten)
- Renders the updated result on result.html with a success message

```
db_plan = _latest_plan(db, data.user_id)
updated_json, _raw = update_workout_plan(
    db_plan.original_plan, data.feedback,
    db_user.name, db_user.age, db_user.weight, db_user.goal, db_user.intensity)

db_plan.updated_plan = updated_json
db_plan.feedback = data.feedback
db_plan.updated_at = utcnow()
db.commit()
return _render_result(request, db_user, db_plan, just_updated=True)
```

/view-all-users – Admin view

- Loads all users and all plans from the database
- Applies the optional search text (name or user ID)
- Computes total users, total plans, updated plans and goal distribution
- Passes everything to all_users.html

Error handling

Missing API key, invalid API key, quota limits, network failures, invalid input, malformed AI responses and database errors all lead to a readable message on error.html. Stack traces are never shown to the user.

Behaviour notes

- Each Generate Plan submit creates a new plan row; feedback always applies to the user's most recent plan.
- Submitting feedback again replaces the previous update (and its feedback text) for that plan.

- There is no login. Anyone who can open the site can see /view-all-users, so the project is meant for a local machine or trusted network.

Milestone 4. Frontend Development

Milestone 4 develops a user-friendly and visually structured interface using HTML, CSS, vanilla JavaScript and Jinja2 templating. Each template is rendered dynamically through FastAPI so that users can submit input, receive plans, give feedback, and administrators can review data.

Activity 4.1: Designing and Developing the User Interface

Base HTML structure

- base.html holds the shared layout; the other pages extend it using Jinja2 template inheritance
- index.html is the entry point and captures name, user_id, age, weight, goal and intensity
- Every field has a label; goal and intensity are drop-down lists filled from the backend
- HTML5 validation (required, min, max, pattern) gives an early check before the server validates again

Responsive layout using CSS

- A single stylesheet, static/css/style.css, is shared by all pages
- Card-based layout with padding, shadows and rounded corners
- The 7-day plan is shown as a grid of day cards; media queries adapt the layout for mobile screens
- Consistent primary and secondary buttons with hover effects

Small JavaScript enhancement

static/js/script.js disables the submit button and shows "Please wait... generating with AI" while a Gemini request is running, and resets the button if the user comes back with the browser's Back button.

Separate pages for each core functionality

Template	Purpose
index.html	User input form (name, user ID, age, weight, goal, intensity)
result.html	User profile, 7-day plan as day cards, nutrition tip, and feedback form
feedback.html	Dedicated feedback page with example feedback suggestions
all_users.html	Admin dashboard: statistics, search, original and updated plans
error.html	Friendly error message page

Activity 4.2: Creating Dynamic Templates with FastAPI's Jinja2

Jinja2 templating for dynamic content

```
from fastapi.templating import Jinja2Templates
BASE_DIR = Path(__file__).resolve().parent.parent
templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))
```

An absolute path is used so the application works no matter which folder the server is started from. Each route calls `TemplateResponse` with the request, template name and context:

```
return templates.TemplateResponse(request, "result.html", {
    "user": db_user,
    "plan": db_plan,
    "display_plan": display_plan,
    "is_updated": bool(db_plan.updated_plan),
    "just_updated": just_updated,
})
```

Bind backend data to HTML templates

index.html

- Form elements use `name="..."` to bind to the backend `Form(...)` parameters
- The form is submitted with POST to `/generate-workout`

result.html

- Shows user data with `{{ user.name }}`, `{{ user.goal }}`, `{{ user.intensity }}` and so on
- Loops over `display_plan.days` to show each day's focus, warm-up, exercises, sets/reps, rest and cooldown
- Shows `{{ plan.nutrition_tip }}` and a feedback form that submits by POST to `/submit-feedback`
- Shows a success alert after a feedback update, and the raw AI text as a fallback if the response could not be parsed

all_users.html

- Shows statistics boxes and the search form
- Loops over all rows to list users and their plans

```
{% for day in display_plan.days %}
<div class="card day-card">
  <h3>{{ day.day }}</h3>
  <p class="focus">Focus: {{ day.focus }}</p>
  <dl>
    <dt>Warm-up</dt><dd>{{ day.warm_up }}</dd>
    <dt>Exercises</dt><dd>{{ day.exercises }}</dd>
    <dt>Sets / Reps / Duration</dt><dd>{{ day.sets_reps }}</dd>
    <dt>Rest</dt><dd>{{ day.rest }}</dd>
    <dt>Cooldown</dt><dd>{{ day.cooldown }}</dd>
  </dl>
</div>
{% endfor %}
```

Milestone 5. Deployment

This milestone deploys FitBuddy on a local Windows system using FastAPI and Uvicorn. It ensures that all dependencies are installed correctly and that the application runs smoothly on a local server.

Activity 5.1: Preparing the Application for Local Deployment

Set up a virtual environment

- Create a virtual environment to isolate the project's dependencies
- Activate it and install everything from requirements.txt

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Configure environment variables

The Gemini API key and model names are stored in a `.env` file in the project folder and loaded with `python-dotenv`. The key is never written in the source code and `.env` is excluded from Git through `.gitignore`. A key can be created at Google AI Studio (aistudio.google.com/apikey).

```
GOOGLE_API_KEY=your_gemini_api_key_here
GEMINI_WORKOUT_MODEL=gemini-2.5-flash
GEMINI_TIP_MODEL=gemini-2.5-flash
GEMINI_FALLBACK_MODELS=gemini-2.5-flash
```

Variable	Purpose
GOOGLE_API_KEY	Gemini API key (required)
GEMINI_WORKOUT_MODEL	Model for the 7-day plan and feedback revisions
GEMINI_TIP_MODEL	Model for the short nutrition / recovery tip
GEMINI_FALLBACK_MODELS	Comma-separated models tried automatically if a model returns 404

One-click run using run.bat

Double-clicking `run.bat` automatically: (1) checks that Python is installed, (2) creates the virtual environment if missing, (3) activates it, (4) installs dependencies if any is missing, (5) creates `.env` from `.env.example` if needed and warns if the API key is still the placeholder, and (6) starts the Uvicorn server.

Activity 5.2: Testing and Verifying Local Deployment

Access the application locally

Start the server with `uvicorn app.main:app --reload` (or `run.bat`). Open `http://127.0.0.1:8000` in a browser for the main interface, and `http://127.0.0.1:8000/docs` for the interactive Swagger documentation. The database file `fitbuddy.db` is created automatically on first start.

Suggested test input

Field	Example value
Name	Priya Sharma
User ID	student001
Age	21
Weight (kg)	58
Fitness Goal	Muscle Gain
Workout Intensity	Medium

Exploring the web pages

Home page

The screenshot shows the FitBuddy web application. At the top, there's a navigation bar with the FitBuddy logo and links for 'Home' and 'Admin Dashboard'. Below this is a large orange banner with the text 'FitBuddy – AI Fitness Plan Generator' and a subtitle 'Generate a personalized 7-day workout plan and nutrition guidance using AI.' The main content area contains a form titled 'Tell us about yourself'. The form has the following fields: 'Name' (text input with placeholder 'e.g. Priya Sharma'), 'User ID' (text input with placeholder 'e.g. student001'), 'Age' (text input), 'Weight (kg)' (text input), 'Fitness Goal' (dropdown menu with 'Endurance' selected), and 'Workout Intensity' (dropdown menu with 'Low' selected). At the bottom of the form is an orange 'Generate Plan' button. The browser window shows the URL '127.0.0.1:8000' and a taskbar at the bottom with various application icons and system status indicators like '35°C Mostly sunny', 'Search', and '14:21 29-09-2028'.

The home page shows the form for Name, User ID, Age, Weight (kg), Fitness Goal and Workout Intensity. Clicking **Generate Plan** sends the data to the backend, which calls Gemini and stores the result.

Personalized workout plan page

The screenshot displays the FitBuddy web application interface. At the top, there's a navigation bar with the FitBuddy logo, a home button, and an admin dashboard link. Below this, the user's profile is shown, including their name (yuva), user ID (yuva123), age (20), weight (66.0 kg), fitness goal (Weight Loss), and workout intensity (Low). The main section is titled "Your 7-Day Workout Plan" and contains seven day cards. Each card lists the focus area, warm-up, exercises, sets/reps/duration, rest, and cooldown. The days are: Day 1 (Full Body Strength), Day 2 (Low-Impact Cardio & Core), Day 3 (Rest / Active Recovery), Day 4 (Full Body Strength), Day 5 (Low-Impact Cardio), Day 6 (Rest / Active Recovery), and Day 7 (Full Body & Flexibility). The interface is clean and modern, with a light gray background and white cards. The bottom of the screen shows a Windows taskbar with various icons and a system tray indicating the date and time.

yuva's Fitness Profile

User ID	Age	Weight	Fitness Goal	Workout Intensity
yuva123	20	66.0 kg	Weight Loss	Low

Your 7-Day Workout Plan

Day 1
Focus: Full Body Strength

Warm-up
5 minutes light cardio (jogging in place, arm circles) and dynamic stretches (leg swings, torso twists).

Exercises
Bodyweight Squats, Incline Push-ups (against a wall or sturdy surface), Reverse Lunges (alternating legs), Plank, Glute Bridges.

Sets / Reps / Duration
2 sets of 10-12 reps for squats, push-ups, lunges, glute bridges. 2 sets of 30-45 seconds for plank.

Rest
60-90 seconds rest between sets.

Cooldown
5 minutes static stretching (quad stretch, hamstring stretch, chest stretch, triceps stretch).

Day 2
Focus: Low-Impact Cardio & Core

Warm-up
5 minutes brisk walking in place and gentle joint rotations.

Exercises
Brisk Walking (outdoors or treadmill at a comfortable pace), Bird-Dog, Crunches, Bicycle Crunches.

Sets / Reps / Duration
30 minutes brisk walking. 2 sets of 10-12 reps per side for Bird-Dog. 2 sets of 15-20 reps for Crunches and Bicycle Crunches.

Rest
45-60 seconds rest between core exercises.

Cooldown
5 minutes light stretching for legs and core.

Day 3
Focus: Rest / Active Recovery

Warm-up
Not applicable, focus on gentle movement.

Exercises
Gentle stretching, foam rolling (if available), or a leisurely 15-20 minute walk.

Sets / Reps / Duration
Perform movements as comfortable, no specific sets/reps.

Rest
Not applicable.

Cooldown
Not applicable.

Day 4
Focus: Full Body Strength

Warm-up
5 minutes light cardio (marching in place, jumping jacks if comfortable) and dynamic arm/leg swings.

Exercises
Wall Sits, Kneeling Push-ups (if comfortable, otherwise Incline Push-ups), Calf Raises, Superman, Standing Side Leg Raises.

Sets / Reps / Duration
2 sets of 30-45 seconds for wall sits. 2 sets of 10-12 reps for push-ups, calf raises, superman, leg raises (per leg).

Rest
60-90 seconds rest between sets.

Cooldown
5 minutes static stretching (calf stretch, back extension stretch, outer hip stretch).

Day 5
Focus: Low-Impact Cardio

Warm-up
5 minutes light cycling or marching in place, gentle leg swings.

Exercises
Stationary Cycling or Elliptical Trainer at a comfortable, steady pace.

Sets / Reps / Duration
30-40 minutes at a low to moderate intensity.

Rest
Not applicable for continuous cardio.

Cooldown
5 minutes static stretching focusing on quads, hamstrings, and glutes.

Day 6
Focus: Rest / Active Recovery

Warm-up
Not applicable, focus on gentle movement.

Exercises
Light yoga flow (beginner level, focusing on gentle stretches and mobility), or a leisurely 20-minute walk in nature.

Sets / Reps / Duration
Perform movements as comfortable, no specific sets/reps.

Rest
Not applicable.

Cooldown
Not applicable.

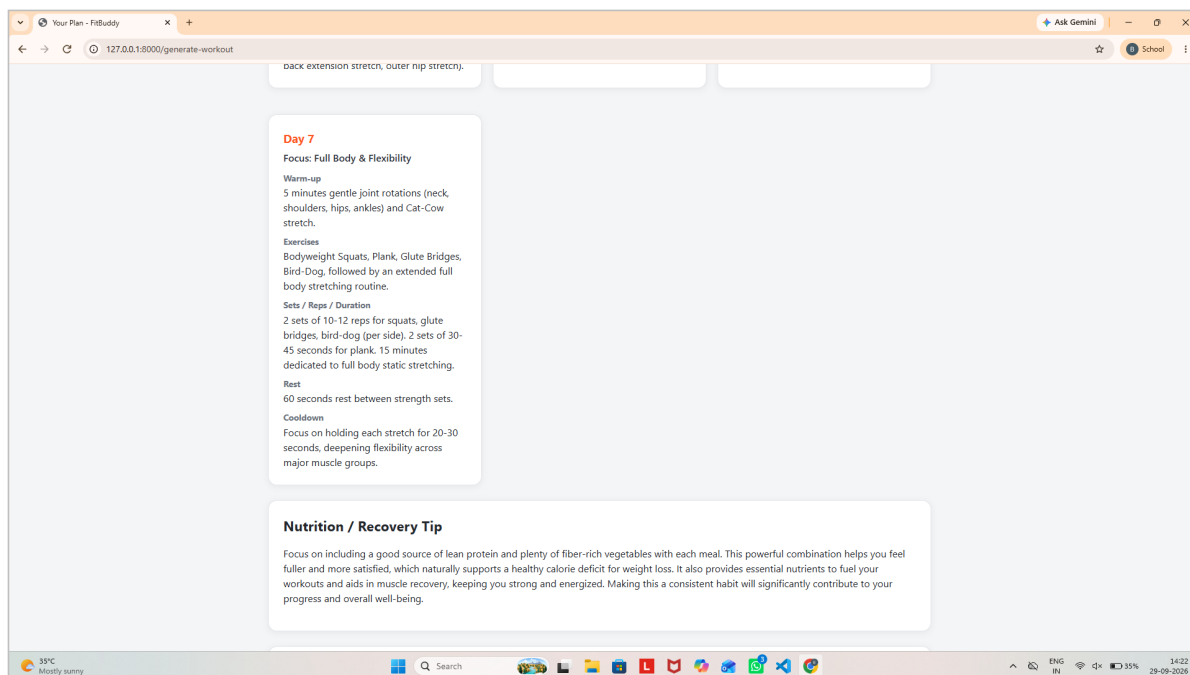
Day 7
Focus: Full Body & Flexibility

Warm-up
5 minutes gentle joint rotations (neck, shoulders, hips, ankles) and Cat-Cow stretch.

Exercises

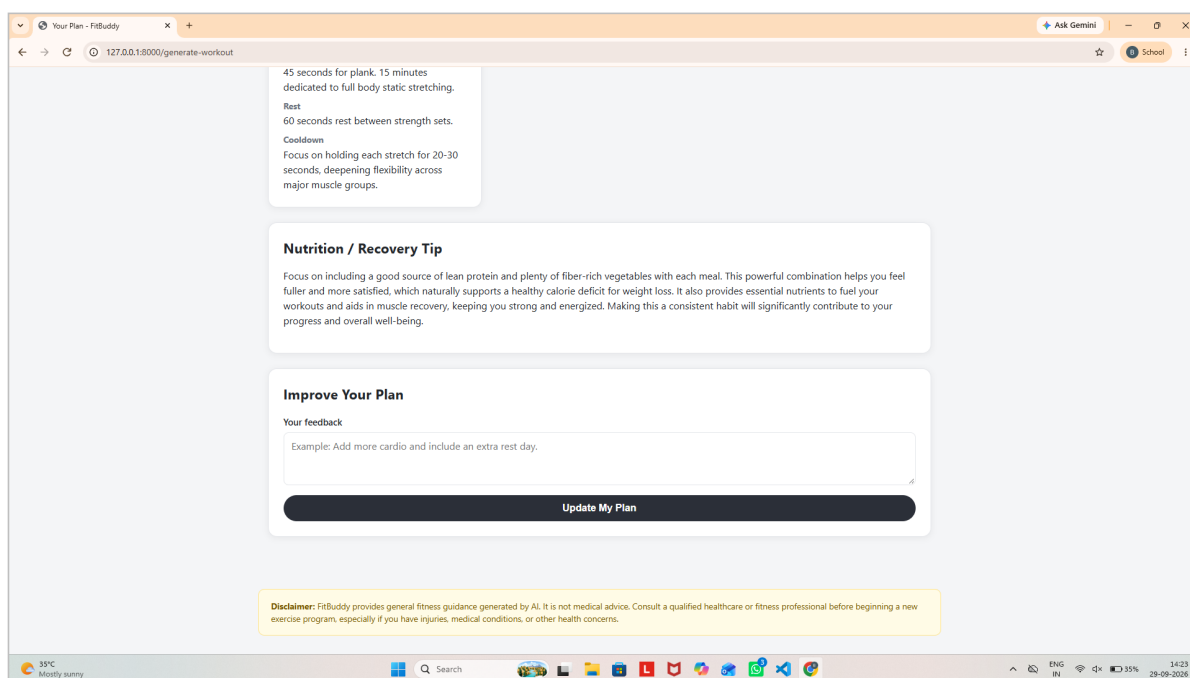
After submission, the result page shows the user's profile and the 7-day plan as day cards. Each card lists the focus area, warm-up, exercises, sets/reps/duration, rest and cooldown.

Nutrition tip



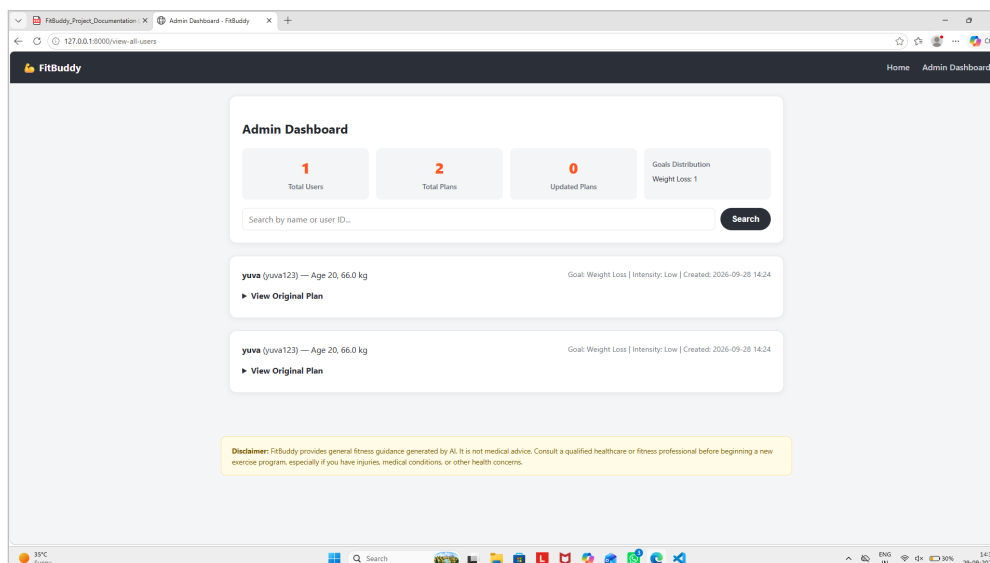
A concise, practical nutrition or recovery tip based on the selected goal is shown beneath the plan.

Feedback



The user enters feedback such as "Add more cardio and include an extra rest day" and clicks **Update My Plan**. Gemini revises the plan, the update is saved, and the page shows the updated plan with a success message.

View all users page



The admin dashboard lists every user with their original plan, updated plan and feedback, together with total users, total plans, updated plans, goal distribution and a search box.

Testing checklist

No.	Test	Expected result
1	Submit the home form with valid data	7-day plan and nutrition tip are shown; data saved in fitbuddy.db
2	Submit an invalid age (for example 5) or invalid goal	Friendly error page, nothing saved
3	Submit feedback for an existing user ID	Updated plan shown with success message; original kept
4	Submit feedback for an unknown user ID	"No user found" error page
5	Open /view-all-users and search by name	Matching users listed with statistics
6	Start the app with a wrong API key	Message that Gemini rejected the API key
7	Open /docs	Swagger UI loads

Troubleshooting

Problem	Fix
GOOGLE_API_KEY is not set	Add the real key to .env and restart the server.
Gemini rejected the API key	Create a new key in Google AI Studio, update .env, restart.
404 / model no longer available	Set GEMINI_WORKOUT_MODEL / GEMINI_TIP_MODEL to a current model in .env.
Quota or rate limit reached	Wait a minute and retry, or use a lighter model.
run.bat says Python was not found	Install Python 3.10+ and tick "Add python.exe to PATH".
Port 8000 already in use	Run: python -m uvicorn app.main:app --reload --port 8001
Changes to .env not applied	Stop the server (CTRL+C) and start it again.

Conclusion:

The FitBuddy – AI Fitness Plan Generator is a complete, user-centric web application that combines artificial intelligence with health-tech ideas. It uses Google's Gemini models to generate personalized 7-day workout plans and short nutrition or recovery tips from inputs such as age, weight, fitness goal and workout intensity.

The application is built with FastAPI for backend logic and API handling, SQLAlchemy with SQLite for persistent storage, Pydantic for validation, and Jinja2 templates with HTML, CSS and JavaScript for a clean, responsive frontend. Its feedback feature lets users describe what they want to change, and the system uses generative AI to update the plan while keeping the original version.

An administrative dashboard shows all registered users and their original and updated plans with simple statistics and search, which suits coaches or institutions managing several fitness profiles. Central error handling, configurable model names and one-click startup with run.bat keep the project reliable and easy to demonstrate.

Overall, FitBuddy demonstrates generative AI integration, REST API design with FastAPI, server-side rendering with Jinja2, ORM-based database access, input validation and a feedback-driven AI regeneration loop in a single, understandable project.

Academic Concepts Demonstrated

- Generative AI / Large Language Models (Google Gemini)
- REST API design with FastAPI
- Server-side rendering with Jinja2 templates
- ORM-based database access with SQLAlchemy (CRUD operations)
- Relational data modelling (User to WorkoutPlan, one-to-many)
- Input validation with Pydantic
- Feedback-driven AI regeneration loop
- Responsive front-end design (HTML, CSS, JavaScript)